

## **Tasks:**

- 1. Install Wireshark and verify that packet capture is working.**
- 2. Start capturing traffic on your active network interface.**
- 3. Perform the following activities while capturing:**
  - **Open Google**
  - **Open YouTube**
  - **Search something on Google**
  - **Visit 3 different websites**
- 4. Stop the capture after 5–10 minutes.**
- 5. Analyze and identify:**
  - **DNS Packets**
  - **TCP Packets**
  - **UDP Packets**
  - **HTTPS Traffic**
- 6. Use the following filters:**
  - **dns**
  - **tcp**
  - **udp**
  - **http**
  - **tls**
- 7. Find and record:**
  - **Source IP Address**
  - **Destination IP Address**
  - **Protocol Used**
  - **Packet Count**
- 8. Take screenshots of:**
  - **Packet Capture Window**
  - **DNS Packets**
  - **TCP Packets**

- **TLS/HTTPS Traffic**
- **Applied Filters**

**Assignment:**

**Prepare a report containing:**

- 1. What is Wireshark?**
- 2. Why is packet analysis important?**
- 3. Difference between TCP and UDP.**
- 4. What is DNS?**
- 5. What is TLS/HTTPS?**
- 6. Screenshots of captured traffic.**
- 7. Explanation of each protocol observed.**

**Capture traffic while:**

- **Opening Gmail**
- **Watching a YouTube video**
- **Performing a Google Search**

**Identify:**

- **DNS Requests**
- **TLS Connections**
- **Number of packets generated**

**Submission:**

**PDF or Word Document with screenshots and findings.**

# Aim

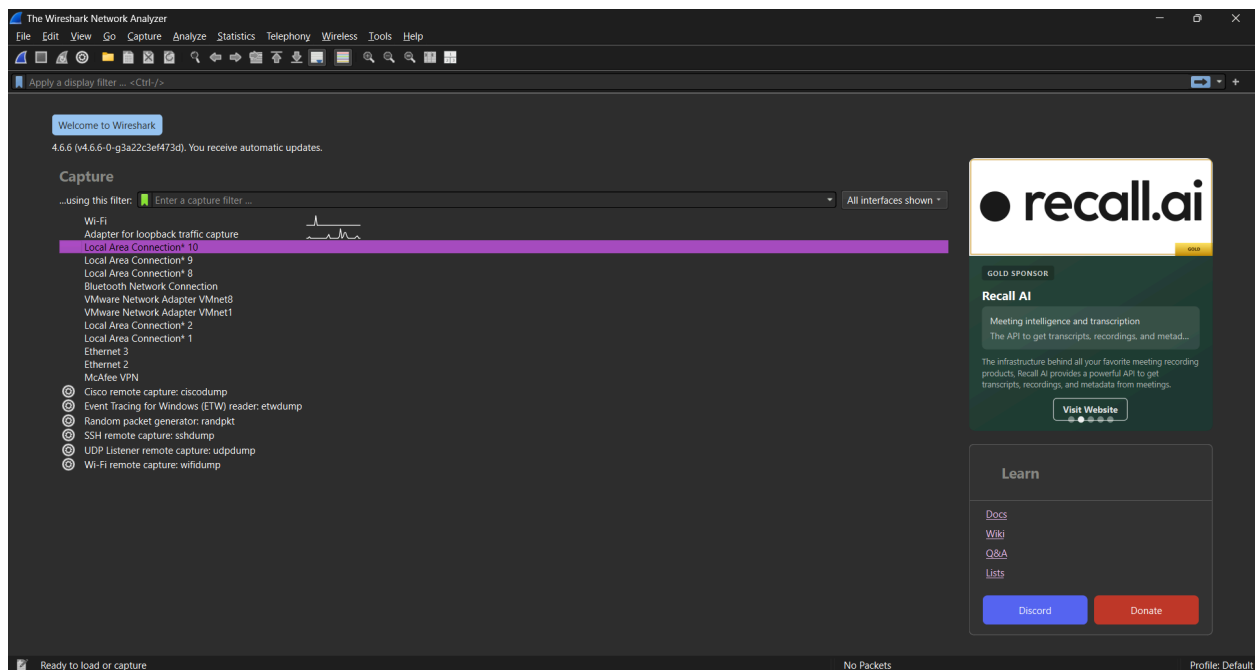
To capture and analyze network traffic using Wireshark and identify different network protocols such as DNS, TCP, UDP, and TLS/HTTPS.

## Objectives

- To install and verify Wireshark packet capturing.
- To capture network traffic generated during common internet activities.
- To identify DNS requests and TLS connections.
- To analyze TCP and UDP traffic.
- To understand the importance of packet analysis.

## What is Wireshark?

Wireshark is a free and open-source network protocol analyzer. It captures packets traveling across a network and allows users to inspect them in detail. It is widely used by network administrators, security professionals, and students for troubleshooting and learning network behavior.



## Why is Packet Analysis Important?

Packet analysis helps in:

- Monitoring network performance.
- Troubleshooting connectivity problems.
- Detecting suspicious or malicious activities.
- Understanding communication between devices.
- Investigating security incidents.

## Difference Between TCP and UDP

| TCP                          | UDP                            |
|------------------------------|--------------------------------|
| Connection-oriented protocol | Connectionless protocol        |
| Reliable data transmission   | No guarantee of delivery       |
| Uses acknowledgments         | Does not use acknowledgments   |
| Slower due to error checking | Faster communication           |
| Used in HTTPS, email, FTP    | Used in DNS, streaming, gaming |

## What is DNS?

DNS (Domain Name System) translates human-readable domain names into IP addresses. For example, when a user enters "[www.google.com](https://www.google.com)", DNS finds the corresponding IP address of the server.

## What is TLS/HTTPS?

TLS (Transport Layer Security) is a cryptographic protocol used to secure communications over a network. HTTPS is HTTP running over TLS, ensuring confidentiality, integrity, and authentication of web traffic.

## **Procedure**

1. Installed Wireshark and verified packet capture.
2. Started capturing traffic on the active Wi-Fi interface.
3. Opened Gmail.
4. Performed a Google search.
5. Opened YouTube and watched a video.
6. Applied filters to analyze captured packets.
7. Identified DNS requests and TLS connections.
8. Recorded packet counts and protocol details.

## **Observations**

### **DNS Requests**

The DNS filter displayed queries and responses for domains such as:

- wikipedia.org
- skype.com
- googleapis.com

Source IP: 10.71.170.76

Destination IP: 10.71.170.94

Protocol: DNS

Displayed DNS Packets: 712

### **TCP Traffic**

TCP packets contained acknowledgements and data transfers.

Example:

Source Port: 443

Destination Port: 56045

Protocol: TCP

Displayed TCP Packets: 8415

## **UDP Traffic**

UDP packets were observed during communication over port 443, indicating the use of protocols such as QUIC.

Displayed UDP Packets: 27453

## **HTTP Traffic**

No HTTP packets were displayed.

Displayed HTTP Packets: 0

This is normal because most modern websites use HTTPS instead of HTTP.

## **TLS/HTTPS Traffic**

TLS traffic showed:

- Client Hello
- Server Hello
- Change Cipher Spec
- Application Data

Example:

TLS Version: TLSv1.3

Displayed TLS Packets: 2479

## **Explanation of Protocols Observed**

### **DNS**

DNS resolves domain names into IP addresses so that browsers can connect to websites.

### **TCP**

TCP provides reliable communication through acknowledgments, sequencing, and retransmissions.

## UDP

UDP provides faster communication without establishing a connection and is used in streaming and QUIC.

## TLS

TLS encrypts data exchanged between clients and servers, ensuring secure communication.

## HTTPS

HTTPS protects web traffic by combining HTTP with TLS encryption.

## Screenshots Included

Figure 1: Wireshark packet capture window.

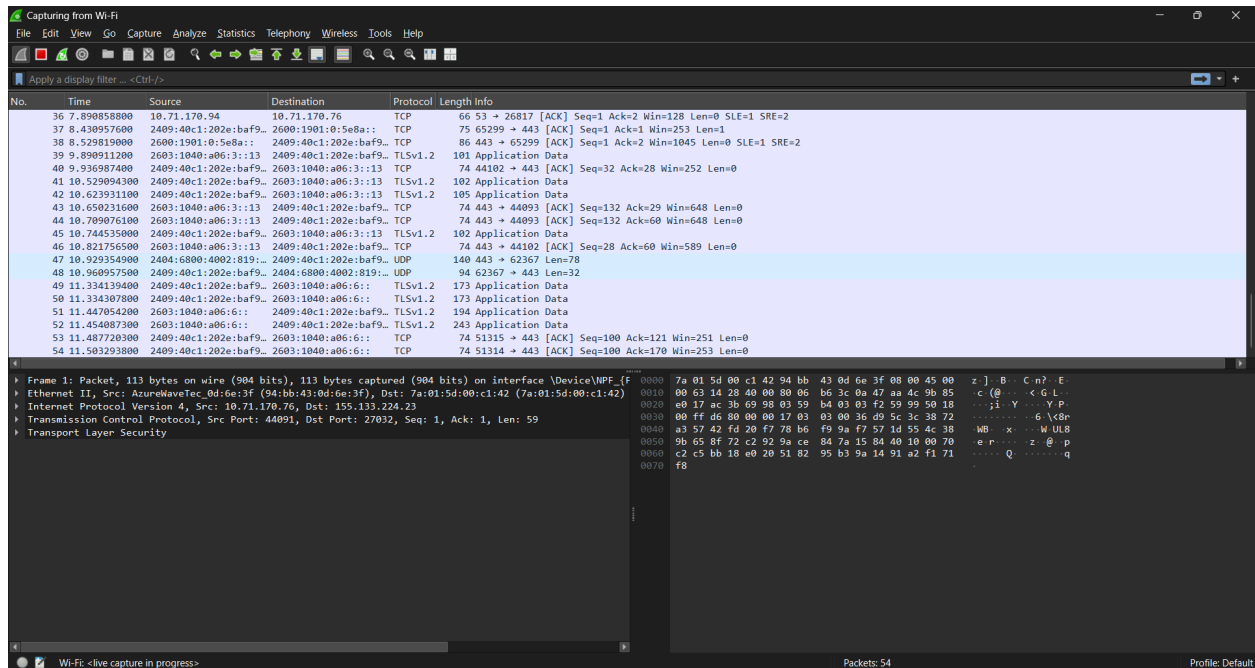


Figure 2: DNS packets using the dns filter.

The image shows a Wireshark packet capture window with the filter 'dns' applied. The packet list shows 13 DNS packets. The packet details pane shows the structure of a DNS query for 'pub-csm-jpwe-02-t.trouter.skype.com'.

| No.   | Time          | Source       | Destination  | Protocol | Length | Info                                                                                                                                |
|-------|---------------|--------------|--------------|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------|
| 10736 | 228.802923400 | 10.71.170.76 | 10.71.170.94 | DNS      | 95     | Standard query 0xcfa8 AAAA pub-csm-jpwe-02-t.trouter.skype.com                                                                      |
| 10737 | 228.803169800 | 10.71.170.76 | 10.71.170.94 | DNS      | 95     | Standard query 0xdc52 A pub-csm-jpwe-02-t.trouter.skype.com                                                                         |
| 10739 | 228.839594400 | 10.71.170.94 | 10.71.170.76 | DNS      | 401    | Standard query response 0xcfa8 AAAA pub-csm-jpwe-02-t.trouter.skype.com CNAME partition-cname-trouter.pub-csm-jpwe-02.ic3-edf-tr... |
| 10740 | 228.841633000 | 10.71.170.94 | 10.71.170.76 | DNS      | 389    | Standard query response 0xdc52 A pub-csm-jpwe-02-t.trouter.skype.com CNAME partition-cname-trouter.pub-csm-jpwe-02.ic3-edf-trout... |
| 10741 | 228.851010600 | 10.71.170.94 | 10.71.170.76 | DNS      | 456    | Standard query response 0xb64e HTTPS pub-csm-jpwe-02-t.trouter.skype.com CNAME partition-cname-trouter.pub-csm-jpwe-02.ic3-edf-t... |
| 10742 | 229.002754800 | 10.71.170.76 | 10.71.170.94 | DNS      | 77     | Standard query 0x3358 A www.wikipedia.org                                                                                           |
| 10748 | 229.002893900 | 10.71.170.76 | 10.71.170.94 | DNS      | 77     | Standard query 0x95db AAAA www.wikipedia.org                                                                                        |
| 10750 | 229.041349100 | 10.71.170.94 | 10.71.170.76 | DNS      | 122    | Standard query response 0x3358 A www.wikipedia.org CNAME dyna.wikimedia.org A 103.102.166.224                                       |
| 10751 | 229.041358400 | 10.71.170.94 | 10.71.170.76 | DNS      | 134    | Standard query response 0x95db AAAA www.wikipedia.org CNAME dyna.wikimedia.org AAAA 2001:df2:e500:ed1a::1                           |
| 10867 | 229.490312700 | 10.71.170.76 | 10.71.170.94 | DNS      | 80     | Standard query 0x867c A upload.wikimedia.org                                                                                        |
| 10868 | 229.490424100 | 10.71.170.76 | 10.71.170.94 | DNS      | 80     | Standard query 0x686a AAAA upload.wikimedia.org                                                                                     |
| 10893 | 229.521975200 | 10.71.170.94 | 10.71.170.76 | DNS      | 108    | Standard query response 0x686a AAAA upload.wikimedia.org AAAA 2001:df2:e500:ed1a::2:b                                               |
| 10895 | 229.528590600 | 10.71.170.94 | 10.71.170.76 | DNS      | 96     | Standard query response 0x867c A upload.wikimedia.org A 103.102.166.240                                                             |
| 11040 | 230.056786000 | 10.71.170.76 | 10.71.170.94 | DNS      | 76     | Standard query 0x951c A en.wikipedia.org                                                                                            |
| 11041 | 230.056885800 | 10.71.170.76 | 10.71.170.94 | DNS      | 76     | Standard query 0x70f7 AAAA en.wikipedia.org                                                                                         |
| 11042 | 230.061146000 | 10.71.170.76 | 10.71.170.94 | DNS      | 91     | Standard query 0xa0e7 A content-autofill.googleapis.com                                                                             |
| 11043 | 230.061218900 | 10.71.170.76 | 10.71.170.94 | DNS      | 91     | Standard query 0xa0c1 AAAA content-autofill.googleapis.com                                                                          |
| 11044 | 230.087096600 | 10.71.170.94 | 10.71.170.76 | DNS      | 121    | Standard query response 0x951c A en.wikipedia.org CNAME dyna.wikimedia.org A 103.102.166.224                                        |
| 11045 | 230.087097400 | 10.71.170.94 | 10.71.170.76 | DNS      | 133    | Standard query response 0x7df7 AAAA en.wikipedia.org CNAME dyna.wikimedia.org AAAA 2001:df2:e500:ed1a::1                            |

Frame 62: Packet, 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface \Device\NPF...  
Ethernet II, Src: AzureWaveTec\_0d:6e:3f (94:bb:43:0d:6e:3f), Dst: 7a:01:5d:00:c1:42 (7a:01:5d:00:c1:42)  
Internet Protocol Version 4, Src: 10.71.170.76, Dst: 10.71.170.94  
User Datagram Protocol, Src Port: 60922, Dst Port: 53  
Domain Name System (query)

Figure 3: TCP packets using the tcp filter.

The image shows a Wireshark packet capture window with the filter 'tcp' applied. The packet list shows 20 TCP packets. The packet details pane shows the structure of a TCP segment with sequence number 443.

| No.  | Time         | Source                 | Destination            | Protocol | Length | Info                                                                                |
|------|--------------|------------------------|------------------------|----------|--------|-------------------------------------------------------------------------------------|
| 3511 | 71.551067400 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 896    | Application Data                                                                    |
| 3512 | 71.551068200 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 434    | Application Data, Application Data                                                  |
| 3513 | 71.551319200 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TCP      | 74     | 51650 → 443 [ACK] Seq=478 Ack=9119 Win=65280 Len=0                                  |
| 3514 | 71.554947900 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TLSv1.3  | 154    | Change Cipher Spec, Application Data                                                |
| 3515 | 71.559776400 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TLSv1.3  | 154    | Application Data                                                                    |
| 3516 | 71.559908000 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TCP      | 1374   | 51650 → 443 [ACK] Seq=638 Ack=9119 Win=65280 Len=1300 [TCP PDU reassembled in 3517] |
| 3517 | 71.559908000 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TLSv1.3  | 340    | Application Data                                                                    |
| 3518 | 71.560166300 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TLSv1.3  | 734    | Application Data                                                                    |
| 3521 | 71.732972100 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 377    | Application Data                                                                    |
| 3522 | 71.732973000 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 377    | Application Data                                                                    |
| 3523 | 71.732974100 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 136    | Application Data                                                                    |
| 3524 | 71.733198700 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TCP      | 74     | 51650 → 443 [ACK] Seq=2872 Ack=9787 Win=64768 Len=0                                 |
| 3525 | 71.736146600 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TLSv1.3  | 105    | Application Data                                                                    |
| 3526 | 71.738183300 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 105    | Application Data                                                                    |
| 3527 | 71.748947800 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TCP      | 74     | 443 → 51650 [ACK] Seq=9818 Ack=2212 Win=70656 Len=0                                 |
| 3528 | 71.751224000 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 109    | Application Data                                                                    |
| 3529 | 71.751389100 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TCP      | 74     | 51650 → 443 [ACK] Seq=2903 Ack=9853 Win=64768 Len=0                                 |
| 3530 | 71.834272900 | 64:ff9b::4e6:ab7c      | 2409:40c1:202e:baf9... | TLSv1.3  | 736    | Application Data                                                                    |
| 3531 | 71.834545400 | 2409:40c1:202e:baf9... | 64:ff9b::4e6:ab7c      | TCP      | 74     | 51650 → 443 [ACK] Seq=2903 Ack=10515 Win=65280 Len=0                                |

Frame 61: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF...  
Ethernet II, Src: 7a:01:5d:00:c1:42 (7a:01:5d:00:c1:42), Dst: AzureWaveTec\_0d:6e:3f (94:bb:43:0d:6e:3f)  
Internet Protocol Version 6, Src: 2404:6808:4002:027::2003, Dst: 2409:40c1:202e:baf9::cd00:c867:4d40:207a  
Transmission Control Protocol, Src Port: 443, Dst Port: 56045, Seq: 1, Ack: 2, Len: 0

Figure 4: UDP packets using the udp filter.

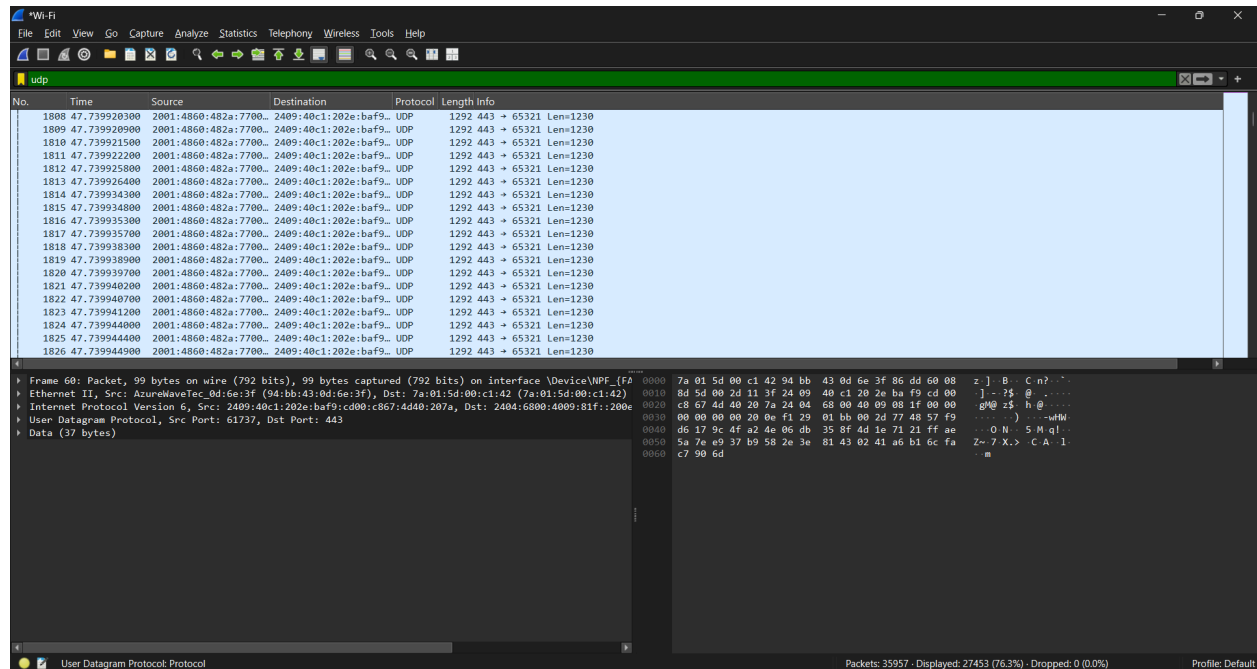


Figure 5: HTTP filter showing no packets.

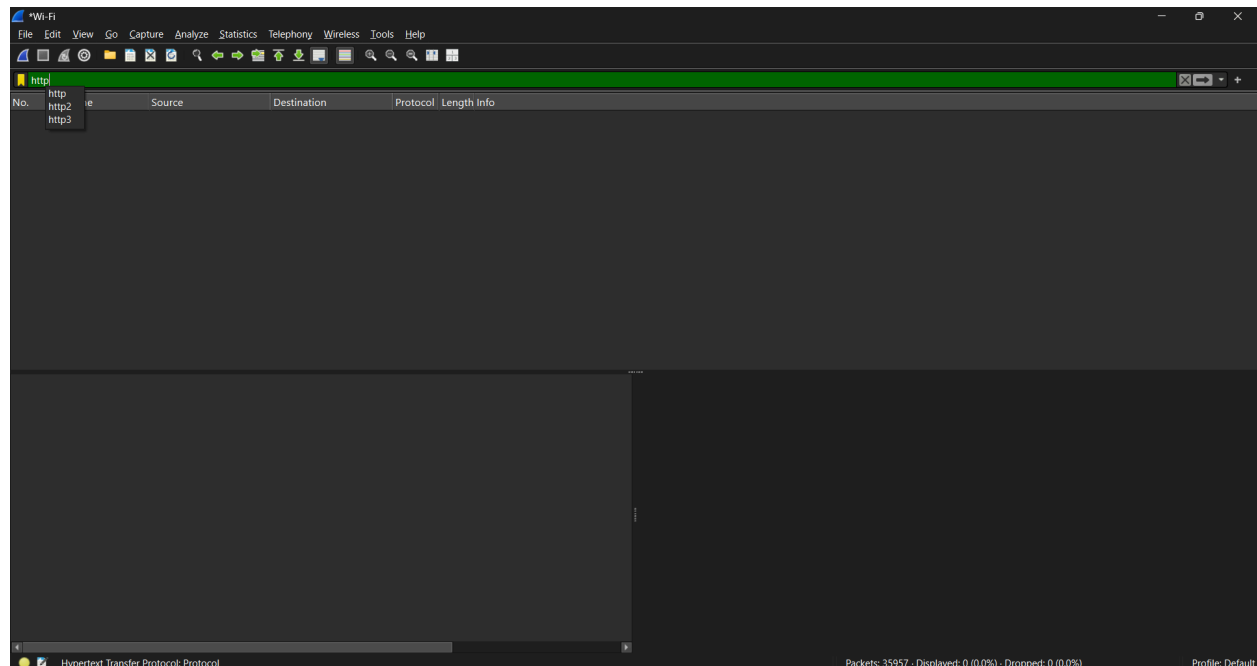
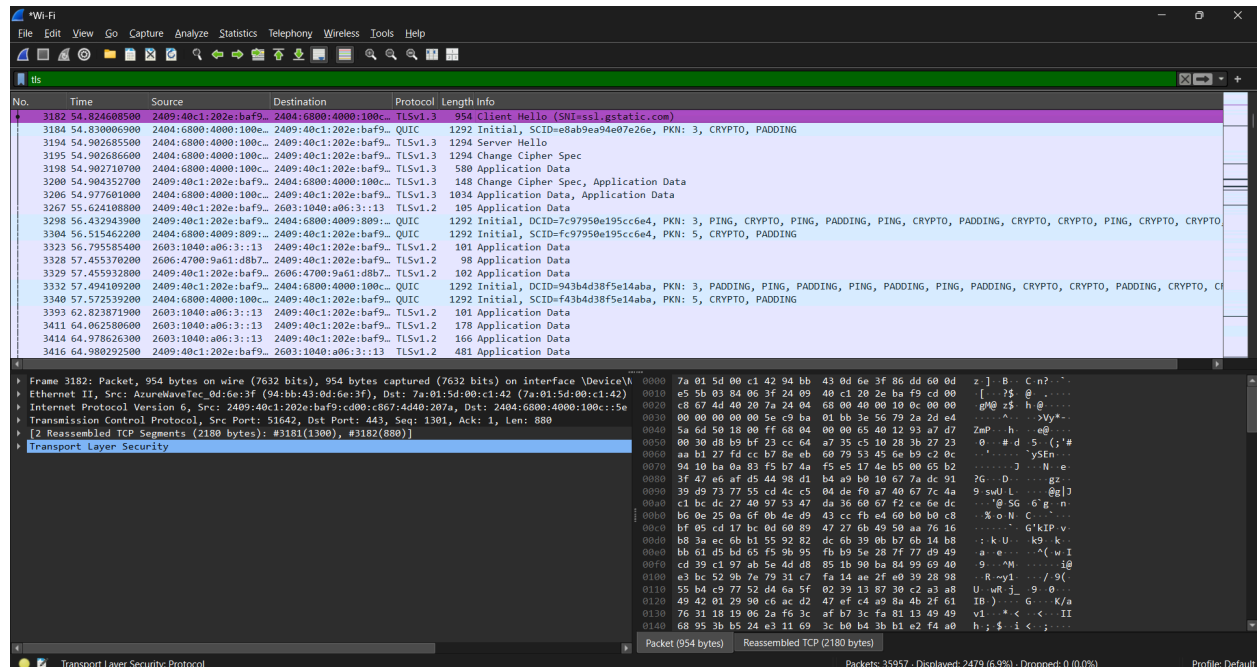


Figure 6: TLS packets using the tls filter.



## Capture traffic while:

- Opening Gmail

While opening Gmail, DNS requests were generated to resolve Gmail-related domains. TLS connections were established to secure communication between the browser and Google's servers. Most of the traffic was encrypted HTTPS traffic carried over TCP.

- Watching a YouTube video

During YouTube video playback, a large number of UDP packets were observed, indicating the use of the QUIC protocol for efficient video streaming. TLS packets were also identified, showing that the communication was encrypted. DNS queries were generated to resolve YouTube-related domains before establishing secure connections.

- **Performing a Google Search**

During the Google search activity, DNS queries resolved Google domains, followed by TLS-secured connections. TCP packets ensured reliable data transfer, and encrypted HTTPS traffic was observed while loading the search results.

## **Result**

Network traffic was successfully captured and analyzed using Wireshark. DNS requests, TCP packets, UDP packets, and TLS-secured communications were identified and examined using display filters.

## **Conclusion**

The experiment demonstrated the practical use of Wireshark in monitoring and analyzing network traffic. It provided an understanding of how DNS resolves domain names, how TCP and UDP operate, and how TLS secures modern internet communications. Packet analysis is an essential technique for network troubleshooting and cybersecurity investigations.